MLDS Assignment 2

Deep Q-Network

A reproducible PyTorch implementation and empirical study of DQN on CartPole-v1 and ALE/MsPacman-v5

GitHub username	Repository	Public URL
fxiong3	mlds-assignment2-dqn	github.com/fxiong3/mlds-assignment2-dqn

Environment	Training	500-episode evaluation	Random baseline
CartPole-v1	1,500 episodes / 183,424 steps	274.21 +/- 214.59	21.9 +/- 10.97 (30 ep.)
MsPacman-v5	6,000 episodes / 3,786,022 steps	1543.58 +/- 232.31	291.0 +/- 155.29 (30 ep.)

MsPacman reached a final 100-episode training average of 1553.5 and a 500-episode evaluation mean of 1543.58. In a 30-episode comparison, it scored 1201 points more than a random policy on average. CartPole performed better than random play but did not meet the 500-step reliability criterion; its 500-episode mean was 274.21.

1. Reproducibility and implementation

The implementation uses an online Q-network, a target network, uniform replay memory, epsilon-greedy exploration, Huber loss, Adam, gradient clipping, and periodic target updates. All reported runs use seed 42. CartPole ran on CPU. MsPacman used one NVIDIA GeForce RTX 2080 Ti on Deepdish3 with PyTorch 2.6.0+cu124.

Network architectures

Task	Input	Architecture	Output
CartPole	4-D float state	MLP: 4 -> 128 ReLU -> 128 ReLU	2 Q-values
MsPacman	4 x 88 x 80 int8 frames	Conv 32(8/4) -> 64(4/2) -> 64(3/1) -> FC 512	9 Q-values

Hyperparameters

Parameter	CartPole-v1	MsPacman-v5
Discount factor	0.95	0.99
Learning rate	1e-3	1e-4
Batch / replay capacity	128 / 50,000	32 / 50,000
Warm-up steps	1,000	20,000
Train frequency	Every step	Every 4 steps
Target sync	500 environment steps	10,000 environment steps
Epsilon	1.0 -> 0.02 over 25k steps	1.0 -> 0.05 over 1M steps
Advanced stabilization	Huber loss, grad clip 10	Double DQN, Huber loss, grad clip 10

How to run

Deepdish setup and tests:

```
ssh vll0756@mlds-deepdish3.ads.northwestern.edu
cd ~/mlds-assignment2-dqn
bash scripts/setup_deepdish.sh
.venv/bin/python -m pytest
.venv/bin/ruff check .
```

Training and required evaluation:

```
bash scripts/run_cartpole.sh
GPU_ID=0 bash scripts/run_mspacman.sh
GPU_ID=0 bash scripts/evaluate_all.sh
```

Before selecting a GPU, inspect nvidia-smi. The launch wrapper refuses to use a card with a compute process, at least 2 GiB allocated, or at least 10% utilization. Long jobs run inside tmux so an SSH disconnect does not interrupt training.

2. CartPole-v1 results

CartPole used gamma=0.95 and a 500-step cap. Training was resumed from episode 600 to episode 1500 after the first run plateaued. The replay buffer is intentionally not serialized, so replay was refilled after resuming. The final checkpoint produced a 500-episode greedy mean of 274.208 (population SD 214.587, minimum 35, maximum 500).

The agent often reached 500 steps, but short failures remained common. The run therefore did not solve CartPole reliably.

Across the first versus last 100 episodes, average reward rose from 21.59 to 139.79 and mean maximum Q rose from 0.911 to 20.276. The best trailing-100 training reward was 260.99, which confirms learning but also shows that the strict 500-step objective was never stable.

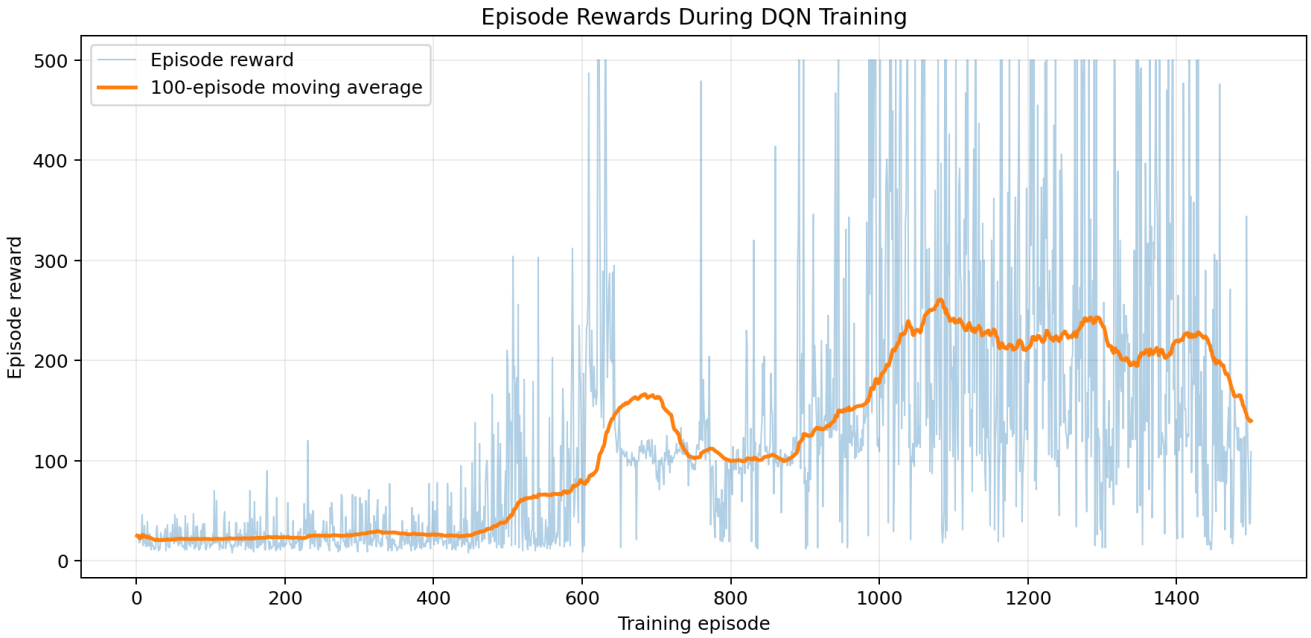


Figure 1. CartPole episode rewards with the trailing 100-episode moving average.

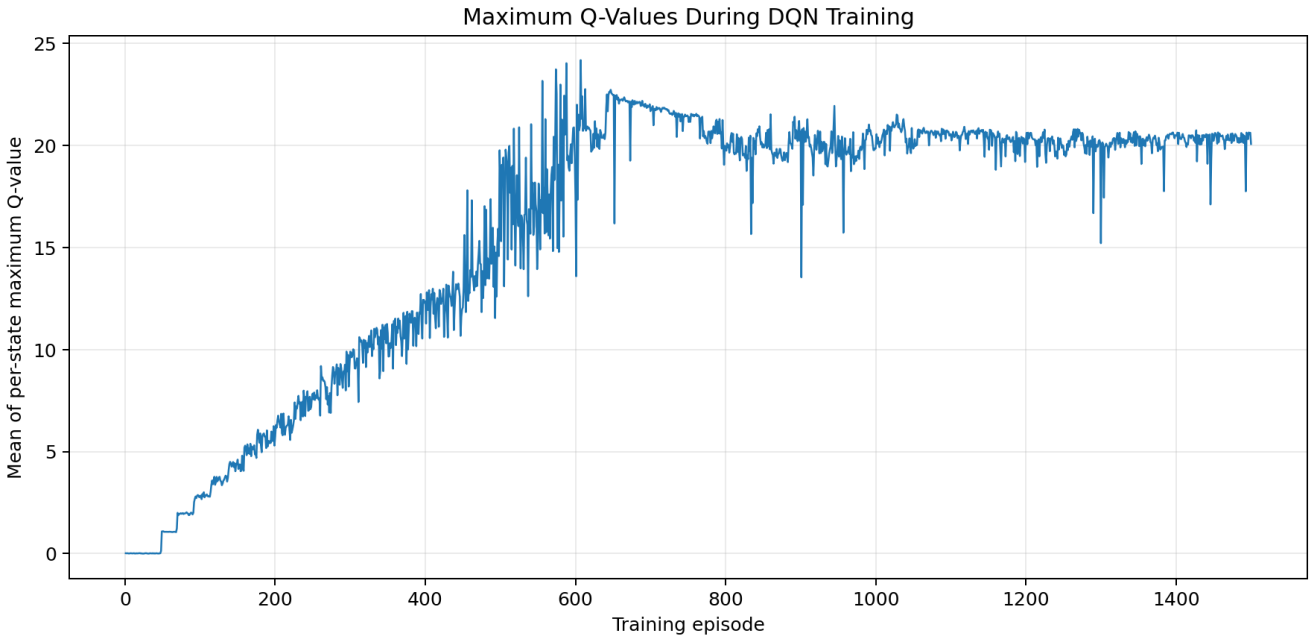


Figure 2. CartPole mean maximum Q-value per training episode.

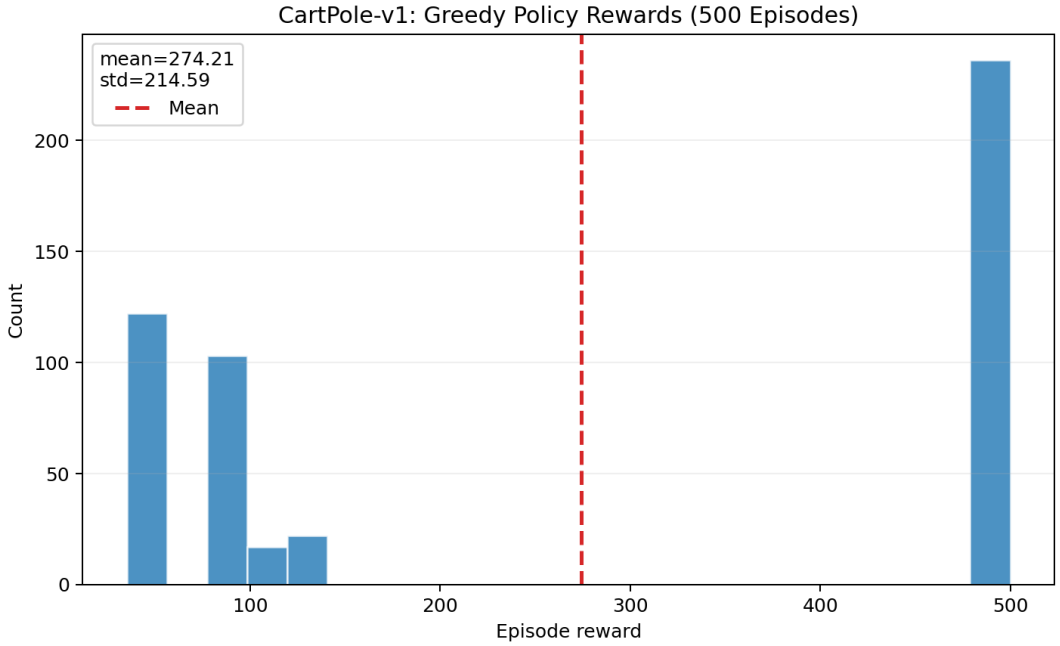


Figure 3. Reward distribution across 500 greedy CartPole evaluation episodes (mean 274.21, SD 214.59).

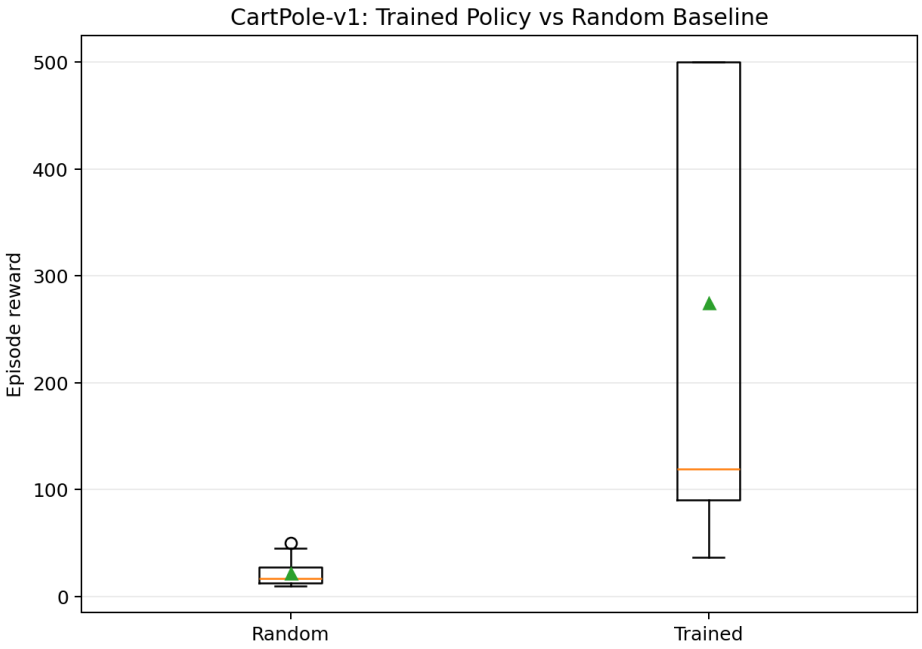


Figure 4. CartPole trained policy versus random actions over 30 matched-seed episodes.

The matched-seed validation gave 274.9 +/- 211.67 for the trained policy and 21.9 +/- 10.97 for random actions, an average improvement of 253.0. The large standard deviation confirms that the trained policy remained sensitive to the initial trajectory.

First ten greedy decisions

Step	Action	max Q	Reward	Terminated	Truncated
0	0	20.524	1.0	False	False
1	0	20.502	1.0	False	False
2	1	20.497	1.0	False	False
3	0	20.493	1.0	False	False
4	1	20.502	1.0	False	False
5	1	20.497	1.0	False	False
6	0	20.508	1.0	False	False
7	1	20.503	1.0	False	False
8	0	20.501	1.0	False	False
9	1	20.511	1.0	False	False

CartPole diagnosis

The mixture of 500-step successes and short failures indicates an unstable policy rather than a failure to learn. Likely causes include the epsilon schedule, hard target updates, and the empty replay buffer after resume. A follow-up run should tune these settings and select the checkpoint on held-out seeds.

3. MsPacman-v5 results

MsPacman used the assignment crop, two-fold spatial downsampling, grayscale conversion, contrast removal, and int8 normalization. Four consecutive frames were stacked to encode motion. The nine-action convolutional agent used gamma=0.99 and Double DQN targets. Training completed 6000 episodes and 3,786,022 environment steps in approximately 4 h 20 min.

The trailing-100 reward increased from 260.0 at episode 100 to 1553.5 at episode 6000, a 5.98x increase. The 500-episode greedy evaluation was 1543.58 +/- 232.31.

The mean per-episode maximum Q-value increased from 0.115 over the first 100 episodes to 119.066 over the last 100. Because the trailing reward rose in parallel and the independent greedy evaluation stayed high, this increase is consistent with learning higher-value states rather than unsupported Q-value inflation.

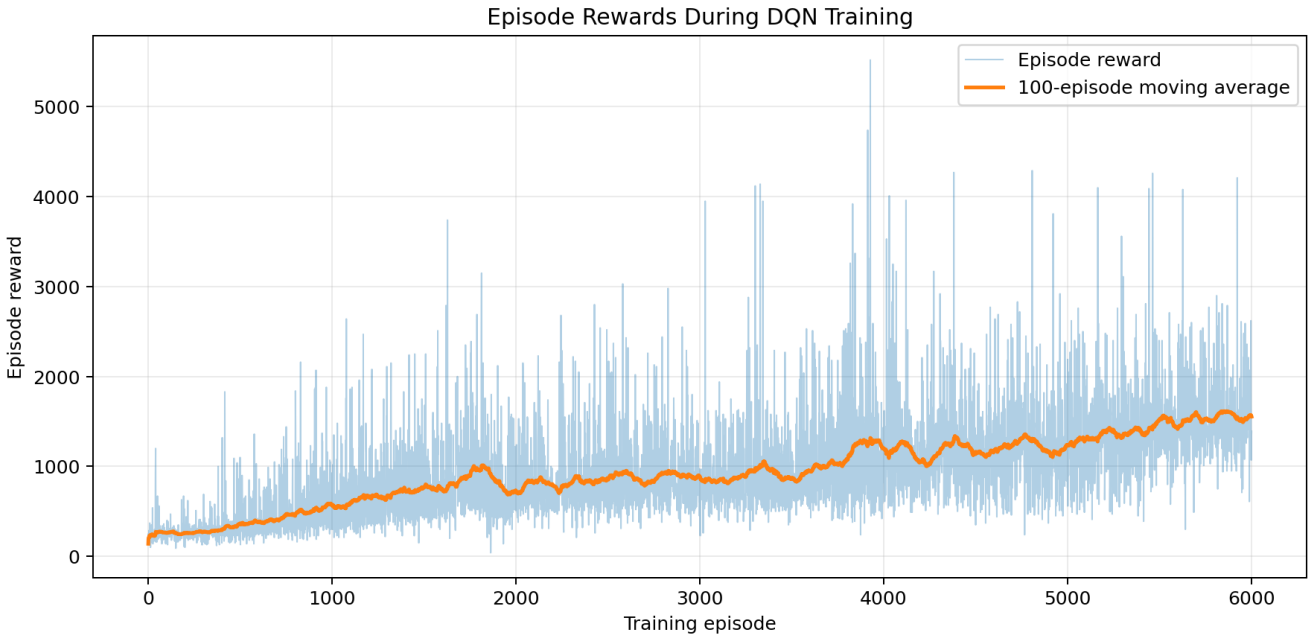


Figure 5. MsPacman training rewards and trailing 100-episode average over 6000 episodes.

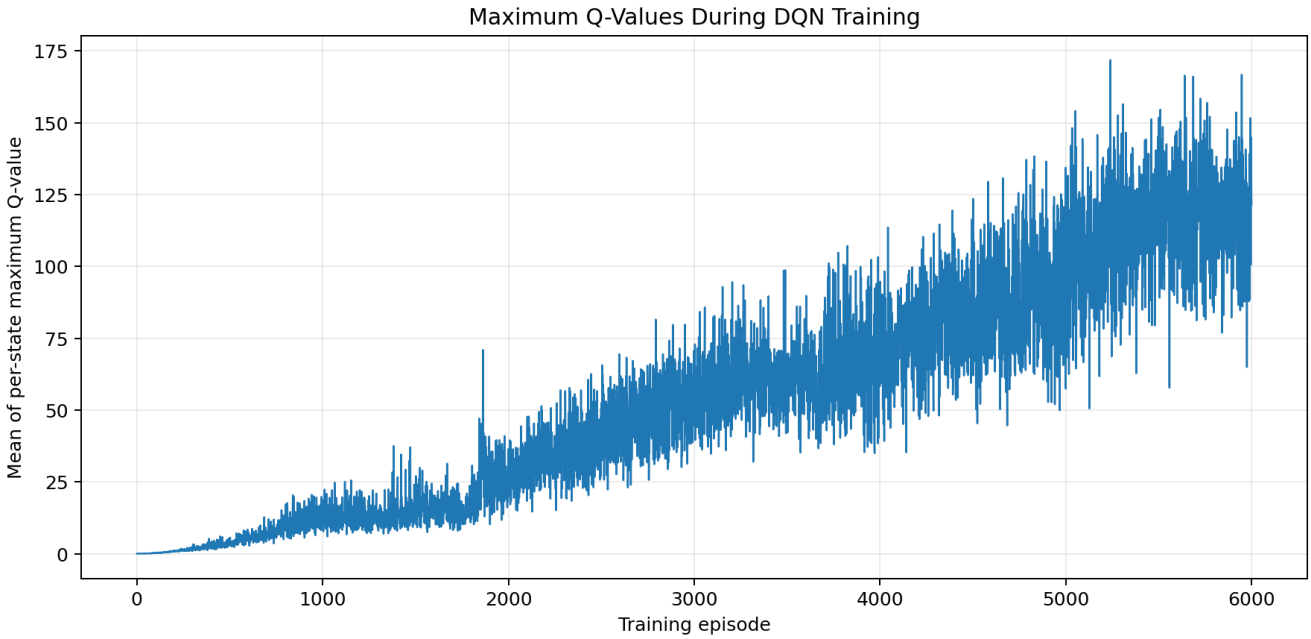


Figure 6. MsPacman mean maximum Q-value per episode; interpreted together with reward to detect value drift.

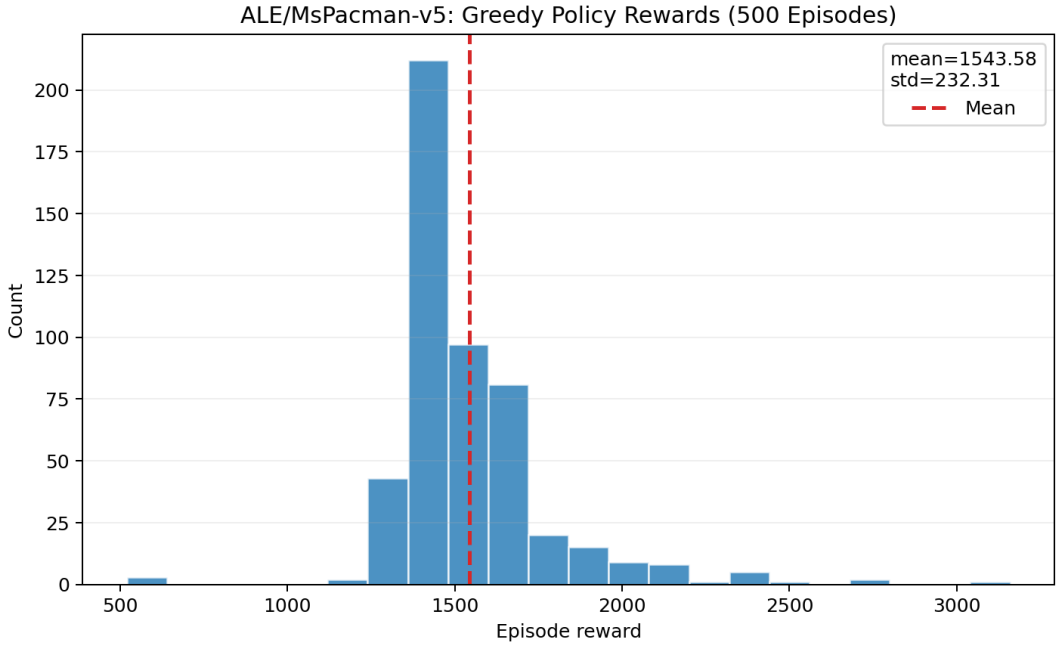


Figure 7. Reward distribution across 500 greedy MsPacman episodes (mean 1543.58, SD 232.31).

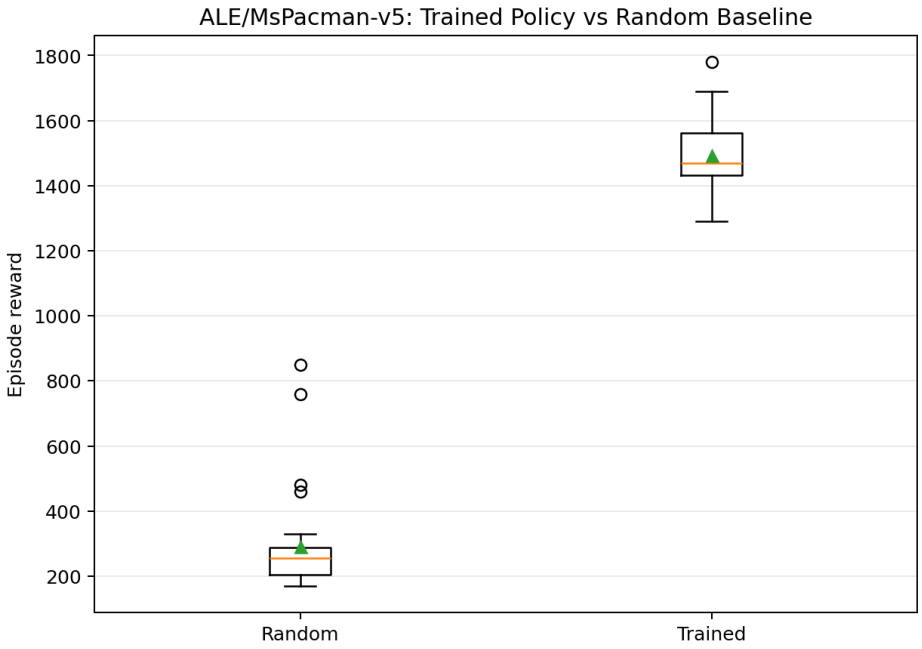


Figure 8. MsPacman trained policy versus random actions over 30 matched-seed episodes.

In the independent 30-episode comparison, the trained policy scored 1492.0 +/- 109.65 versus 291.0 +/- 155.29 for random actions. The mean improvement was 1201 points, or 5.13x the random-policy score. The trained policy also had lower variance.

First ten greedy decisions

Step	Action	max Q	Reward	Terminated	Truncated
0	7	0.734	0.0	False	False
1	7	0.805	0.0	False	False
2	7	0.865	0.0	False	False
3	7	0.793	0.0	False	False
4	7	0.577	0.0	False	False
5	1	0.685	0.0	False	False
6	7	0.618	0.0	False	False
7	7	0.764	0.0	False	False
8	7	0.670	0.0	False	False
9	7	0.824	0.0	False	False

4. Non-standard analysis: action-frequency dynamics

The required non-standard evidence is a 100-episode moving action-frequency heatmap. Each row is one discrete action and each column is a training episode; color represents the action's share of all decisions in the local 100-episode window. Unlike a reward curve, this plot exposes policy collapse, unused actions, and behavioral phase changes.

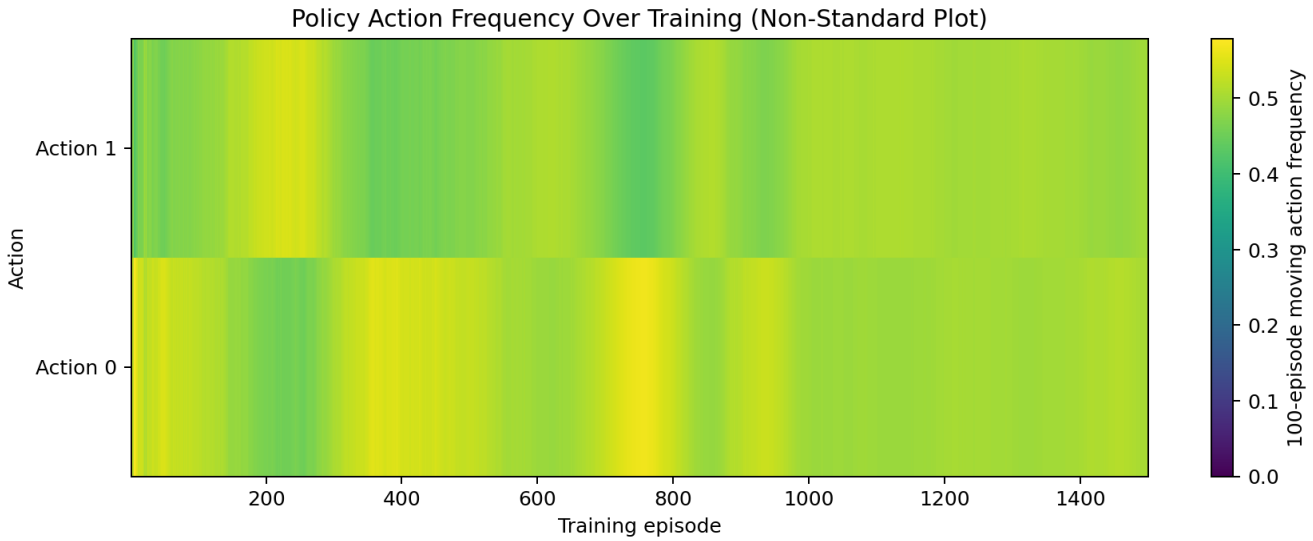


Figure 9. CartPole action-frequency heatmap. A healthy controller should use both left and right actions rather than collapse permanently to one direction.



Figure 10. MsPacman action-frequency heatmap across nine actions. Persistent diversity together with rising reward distinguishes specialization from total action collapse.

The heatmaps should be interpreted jointly with reward and Q-value curves. A single dominant action can be appropriate, but global collapse together with falling reward would indicate a problem. CartPole remained balanced: action 0 changed from 51.23% in the first 100 episodes to 49.88% in the last 100, while action 1 changed from 48.77% to 50.12%. MsPacman began almost uniform (each action between 10.97% and 11.38%) and specialized without collapsing: in the final 100 episodes, all nine actions remained active with frequencies from 7.93% to 16.12%. This behavioral specialization coincided with a reward increase from 260.0 to 1553.5.

5. Edge cases and correctness validation

Risk / edge case	Handling and evidence
Terminal vs. time-limit end	Only true termination zeros the Bellman bootstrap. Truncation keeps the next-state value; CartPole reaching 500 steps is therefore a success, not a failure.
Replay underflow / wraparound	Optimization begins only after one complete batch is present. Replay capacity is fixed; sampled tensors preserve compact int8 Atari storage.
Target-network drift	Target weights are synchronized at initialization and on a fixed environment-step schedule; target parameters are frozen.
Atari temporal ambiguity	Four frames are stacked; the first frame is repeated four times after reset so shape and temporal context are well-defined.
Invalid observations or action spaces	MsPacman input shape must be 210 x 160 x 3 and the environment must expose exactly nine discrete actions; violations fail fast.
Evaluation leakage	Evaluation uses epsilon=0 and a distinct deterministic seed range. Random and trained baselines use matched seeds.
Interrupted SSH / training	tmux preserved the process. Checkpoints store online/target networks, optimizer, episode, and step counters; metrics are retained across resume.
Partial checkpoints	Checkpoint data are written to a temporary file and atomically renamed, preventing a partially written final file.

Validation strategy beyond unit tests

1. 500-episode greedy rollout: measures the delivered policy without exploration and reports full reward distributions, not a cherry-picked episode.
2. Matched-seed random baseline: establishes that learned performance is caused by the network policy rather than easy initial states.
3. First-ten-step trace: logs state statistics, chosen action, max Q, reward, and both termination flags, allowing local inspection of state-value-policy interaction.
4. Q/reward co-monitoring: checks that increasing Q estimates correspond to reward improvement instead of unbounded value inflation.
5. Action-frequency heatmap: detects policy collapse and behavioral phase changes.

The automated suite contained 41 tests and achieved 81.57% statement coverage. It checks Bellman terminal masking, target synchronization, replay wraparound/dtypes, image preprocessing, network shapes, metric persistence, plotting, checkpoint loading, evaluation, and an end-to-end CartPole smoke run. These are engineering checks; the rollout and baseline comparisons above are the task-level algorithm validation required by the assignment.

6. Limitations and three-month extension plan

Current limitations

Only one training seed was completed, so the report does not claim confidence intervals over training randomness. CartPole did not meet the strict solved criterion and should be treated as a partial result. Checkpoint resume restores optimizer and model state but not replay contents. MsPacman was evaluated in the training environment configuration only.

Experiments with an additional three months

Experiment	Purpose and decision criterion
Five or more seeds	Report bootstrap 95% confidence intervals for learning curves and 500-episode means; reject improvements whose intervals overlap materially.
DQN vs. Double DQN vs. prioritized replay	Separate overestimation control from sampling efficiency using equal environment-step and wall-clock budgets.
CartPole stability sweep	Grid-search learning rate, target update, epsilon decay, replay size, and soft vs. hard target updates; require at least 475 mean over 100 held-out episodes.
Robustness	Evaluate Atari sticky actions, observation noise/occlusion, frame-drop, and seed shifts; report degradation curves rather than one operating point.
Q calibration	Compare predicted Q values with Monte Carlo returns from replayed states; report bias, RMSE, and reliability plots.
Efficiency profiling	Measure sample efficiency, wall time, GPU utilization, host memory, and energy per reward improvement.
Representation diagnostics	Visualize saliency and action-conditioned embeddings to determine whether the CNN attends to agents, ghosts, pellets, and corridors.

7. Conclusion

The implementation satisfies the core DQN requirements: neural function approximation, replay memory, target networks, assignment-specific discount factors and preprocessing, one-GPU Atari training, required plots, and 500-episode evaluations. MsPacman converged and substantially outperformed random play. CartPole learned but remained unstable. The next experiments should focus on CartPole stability and replication across multiple seeds.